Project Sprint #2

The Solitaire game is described in CS449HomeworkOverview.docx. You should read the description very carefully.

Your submission must include the GitHub link to your project, and you must ensure that the instructor has the proper access to your project. You will receive no points otherwise.

GitHub link:

In this assignment, you aim to implement the minimum features including **choosing the board size and type, starting a new game**, **making a move, determining if a game is over**. The following is a sample GUI layout.

The implementation of a GUI is required. You should make every attempt to make your code easy to extend. It is required to separate the user interface code and the game logic code into different classes (refer to the TicTacToe example). xUnit tests are required.

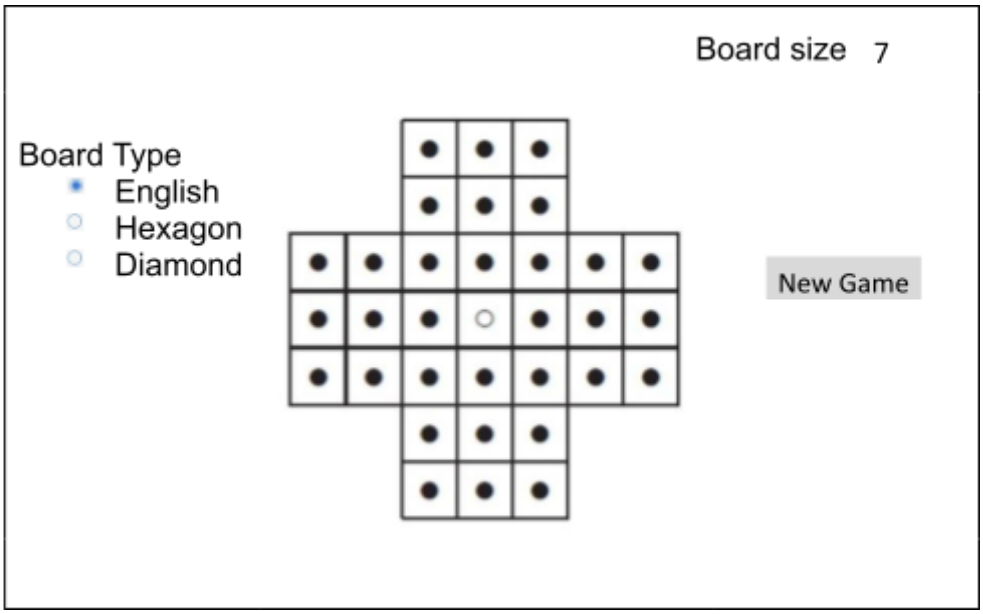


Figure 1. Sample GUI layout of the Sprint 2 program

Deliverables:

1. **Demonstration (8 points)**

   Submit a link to a video of no more than three minutes, clearly demonstrating that you have implemented the required features and written some automated unit tests. In the video, you must explain what is being demonstrated. No points will be given without a video link.
   YouTube/Panopto link:

	Feature
1	Choose board size and type
2	Start a new game of the chosen board size and type

3	Making a move
4	Checking the end of game
5	Automated unit tests

2. Summary of Source Code (1 points)

Source code file name	Production code or test code?	# lines of code
Game_logic.gd	Test code	121
test_board.gd	Test code	105
	Total	226

You must submit all source code to get any credit for this assignment.

3. Production Code vs User stories/Acceptance Criteria (3 points)

Update your user stories and acceptance criteria from the previous assignment (if required) and ensure they adequately capture the requirements. Summarize how each of the following user story/acceptance criteria is implemented in your production code (class name and method name etc.). If you are not using an OOP, then adapt the tables accordingly.

User Story ID	User Story
1	Choose board size and type
2	Start a new game of the chosen board size and type
3	Making a move
4	Checking the end of game

User Story ID and Name	AC ID	Class Name(s)	Method Name(s)	Status (complete or not)	Notes (optional)
1.	1.1	BoardGame	_on_board_size_selection_item_selected()	Complete	
	1.2	BoardGame	_on_board_size_selection_item_selected()	Complete	Functional
	1.3	nothing	nothing	N/A	May not be necessary
2	2.1	OptionButton	_on_board_type_selection_item_selected()	Complete	Comes Built in
	2.2	OptionButton	_on_board_size_selection_item_selected()	Complete	
3	3.1	Control	_on_cell_clicked	Incomplete	
	3.2	Control	_on_cell_clicked	Incomplete	
	3.3	Control	_on_cell_clicked	Incomplete	
4	4.1	Control	_on_new_game_pressed()	Incomplete	
5	5.1	Control	N/A	Incomplete	

6	6.1	BaseButton	_on_new_game_pressed ()	Complete	Functional

4. Tests vs User stories/Acceptance Criteria (3 points)

Summarize how each of the user story/acceptance criteria is tested by your test code (class name and method name) or manually performed tests.

User Story ID	User Story
1	Choose board size and type
2	Start a new game of the chosen board size and type
3	Making a move
4	Checking the end of game

4.1 Automated tests directly corresponding to the acceptance criteria of the above user stories
You are required to use ChatGPT to create at least 2 unit tests. You also need to ensure that the generated test cases are correct and refine them if not. At the end of the submission, provide the screenshots of your ChatGPT prompts and answers, along with errors ChatGPT made and you fixed. You may also use another LLM, including hosted locally. Points will be deducted if no screenshots are provided.

User Story ID and Name	Acceptance Criterion ID	Class Name (s) of the Test Code	Method Name(s) of the Test Code	Status (Pass/Fail/In Progress)
1	1.1	BoardGame	set_board_size()	passed
	1.2	nothing	nothing	N/A
	1.3	nothing	nothing	N/A
2	2.1	BoardGame	N/A	N/A
	2.2	BoardGame	set_board_type()	passed
3	3.1	BoardGame	get_piece_at() make_move()	passed
	3.2	BoardGame	nothing	N/A
	3.3	BoardGame	make_move()	passed
4	4.1	BoardGame	start_new_game()	passed
5	5.1	BoardGame	_check_end_of_game()	passed
6	6.1	Control	nothing	N/A
	6.2	Control	nothing	N/A

4.2 Manual tests directly corresponding to the acceptance criteria of the above user stories (if any)

User Story ID and Name	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
1	1.1			
	1.2			
	…			
2	2.1			
	…			

4.3 Other automated or manual tests not corresponding to the acceptance criteria of the above user stories (if any)

Number	Test Input	Expected Result	Class Name of the Test Code	Method Name of the Test Code

Submission Guidelines:

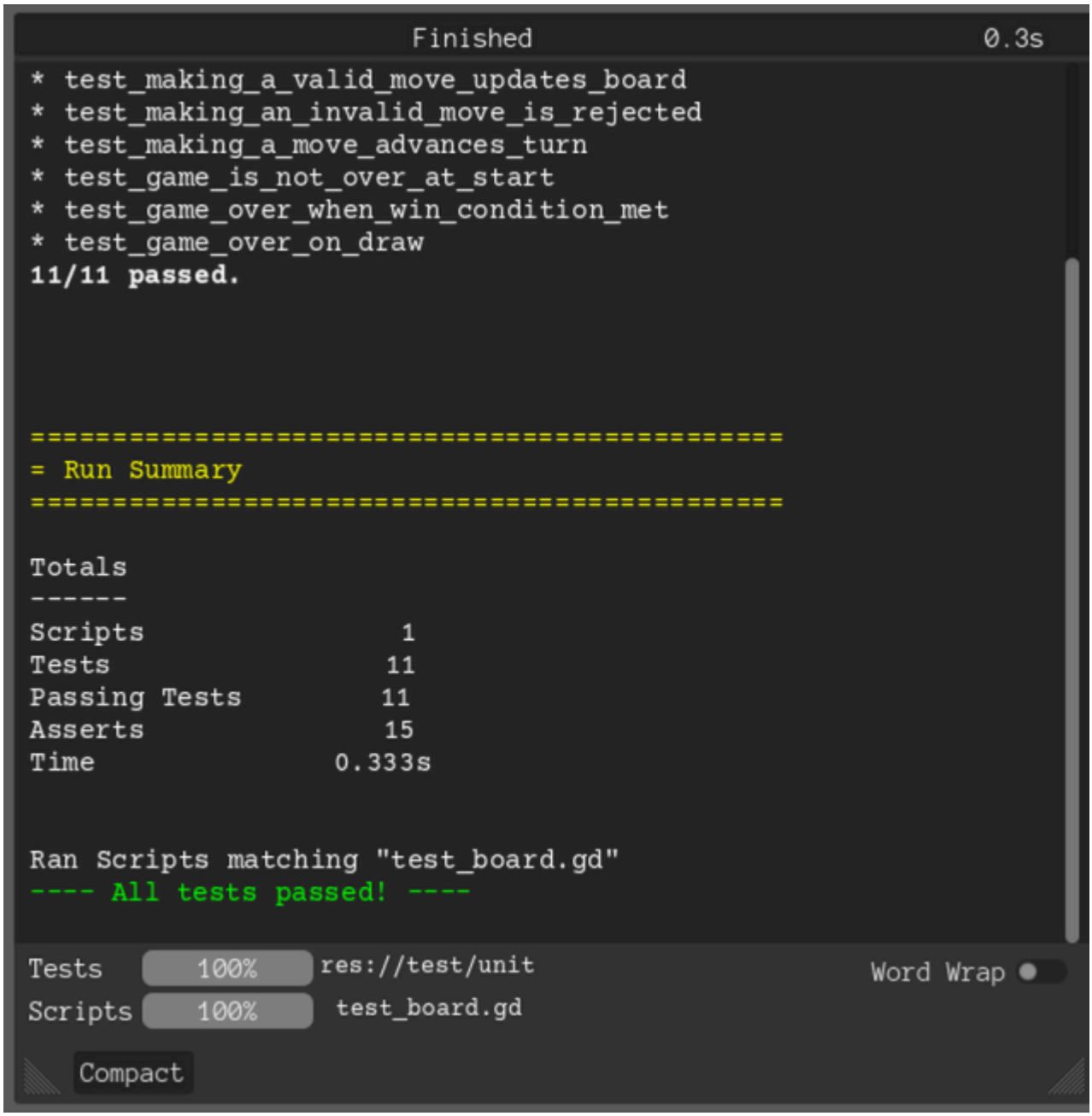
Each student must independently upload **LastName_FirstName_Sprint2.docx** to Canvas. Include:

1. **Your name**
2. **GitHub repository link** containing all production code and automated tests. Ensure that the link is public.
3. **Demonstration video link** — a narrated video of no more than 3 minutes showing the required features and automated tests (again the link should be public).
4. **Source-code summary table** — filenames, production/test classification, line counts, and total.
5. **Updated user stories and acceptance criteria**, mapped to implementation classes and methods, with completion status.
6. **Testing tables** — automated test mappings and results, plus manual or additional tests where applicable.
7. **LLM evidence** — screenshots of prompts and responses used to generate at least two unit tests, with an explanation of how you checked them and corrected any errors.

Mark unused optional sections **"Not applicable."** Before submitting, verify that the instructor can access your GitHub repository and video.

https://chat.deepseek.com/a/chat/s/68495528-370f-4cbf-a665-884cad8c7c97
https://www.youtube.com/watch?v=o2p5WgO8ifc


Finished 0.3s
res://test/unit/test_board.gd
* test_can_select_board_size
* test_can_select_board_type
* test_invalid_board_size_is_rejected
* test_start_new_game_initializes_correct_state
* test_start_new_game_clears_previous_state
* test_making_a_valid_move_updates_board
* test_making_an_invalid_move_is_rejected
* test_making_a_move_advances_turn
* test_game_is_not_over_at_start
* test_game_over_when_win_condition_met
* test_game_over_on_draw
11/11 passed.


Finished
0.3s
* test_making_a_valid_move_updates_board
* test_making_an_invalid_move_is_rejected
* test_making_a_move_advances_turn
* test_game_is_not_over_at_start
* test_game_over_when_win_condition_met
* test_game_over_on_draw
11/11 passed.
= Run Summary
Totals
Scripts 1
Tests 11
Passing Tests 11
Asserts 15
Time 0.333s
Ran Scripts matching "test_board.gd"
---- All tests passed! ----
Tests 100% res://test/unit
Scripts 100% test_board.gd
Word Wrap
Compact

Can u make some sample unit tests in GUT with this?
Choose board size and type
Start a new game of the chosen board size and type
Making a move
Checking the end of game

Found 14 web pages

Here are sample GUT unit tests for a board game, based on the four requirements you listed. The tests assume a typical board game class (e.g., `BoardGame`) with methods for configuration, starting, moving, and checking game over.

Sample GUT Test Script

gdscript Copy Download

```gdscript
# test_board_game.gd
extends GutTest

# Preload the script/class you want to test
const BoardGame = preload("res://src/board_game.gd")

var game: BoardGame

# Runs before EACH test to give a fresh instance
func before_each() -> void:
    game = autofree(BoardGame.new())

# --- 1. Choose board size and type ---

func test_can_select_board_size() -> void:
    game.set_board_size(8)
    assert_eq(game.board_size, 8, "Board size should be set to 8")

func test_can_select_board_type() -> void:
    game.set_board_type("hex")
    assert_eq(game.board_type, "hex", "Board type should be 'hex'")

func test_invalid_board_size_is_rejected() -> void:
    # Example: assuming negative or zero sizes are invalid
    var result = game.set_board_size(-1)
    assert_false(result, "Negative board size should be rejected")
```

Can you write some basic implementations for me to test since "res://src/board_game.gd" does not exist

Here's a basic `BoardGame` implementation that satisfies the tests. It's a simple tic-tac-toe-style game to keep things concrete.

gdscript Copy Download

```gdscript
# res://src/board_game.gd
class_name BoardGame
extends RefCounted

const VALID_BOARD_TYPES := ["square", "hex"]
const VALID_SIZES := [1, 3, 5, 8]

var board_size: int = 3
var board_type: String = "square"
var board: Array = []          # flat array of cell values (null or player id)
var current_turn: int = 1
var is_game_active: bool = false
var winner: String = ""
var _move_count: int = 0

# --- 1. Board configuration ---

func set_board_size(size: int) -> bool:
    if size <= 0 or not VALID_SIZES.has(size):
        return false
    board_size = size
    return true

func set_board_type(type: String) -> bool:
    if not VALID_BOARD_TYPES.has(type):
        return false
    board_type = type
    return true

# --- 2. Start a new game ---

func start_new_game() -> void:
    board.clear()
    for i in range(board_size * board_size):
        board.append(null)
    current_turn = 1
    is_game_active = true
```